EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA

TECNOLÓGICO NACIONAL DE MÉXICO

Tecnológico Nacional de México

Campus Querétaro

E16-Responsable de calidad

Producto de la asignatura:
Ingeniería de Software

Desarrolla por:

Hernandez Perrusquia Jhoel
21140780

Asesor docente:

Villeda Maldonado Julio Alejandro

Fecha de entrega:

20 de marzo de 2026

Grupo:
6A

Validación de coherencia

Lista de inconsistencias detectadas

En esta fase se verificó que cada funcionalidad descrita en los casos de uso tenga un soporte lógico (métodos y atributos) en el diagrama de clases.

- **Inconsistencia en Autenticación:** El caso de uso "Autenticarse en el sistema" es compartido por los actores **Docente**, **Administrativo** e **Instructor** . Sin embargo, en el diagrama de clases, solo la clase **Instructor** posee el método **iniciarSesion().** Las clases **Docente** y **Administrativo** carecen de métodos para gestionar su acceso al sistema .
- **Ausencia de Lógica de Registro:** Existe el caso de uso "Registrar información personal o académica" , pero no se identifica una operación de "creación" o "registro" (ej. **registrarUsuario()**) en la clase base **Usuario** ni en sus subclases. Los métodos existentes se centran en la consulta y no en el alta inicial del perfil.
- **Desconexión Administrativa:** Existe una clase con alta carga de métodos (**AdministradorDDA**) que cuenta con casos de uso definidos, pero que no tiene un controlador o servicio asignado en el diagrama de componentes.
- **Ambigüedad en el Avance:** El componente **InscripcionService** asume la responsabilidad de actualizar horas, pero funcionalmente es un proceso distinto al de la inscripción inicial, lo que genera un riesgo de sobrecarga de funcionalidad.

Problemas de coherencia

Se analizó si la arquitectura modular (componentes) es capaz de alojar y gestionar las entidades definidas en el diagrama de clases.

- **Omisión de Controladores Administrativos:** El diagrama de componentes presenta cinco módulos principales: Auth, Curso, Inscripcion, Constancia y Catalogo. No obstante, la clase **AdministradorDDA** posee una lógica extensa que no parece encajar en ninguno de estos controladores . Se requiere un **AdminController** o integrar explícitamente estas funciones en los componentes existentes.
- **Desconexión de Repositorios:** El diagrama de componentes define repositorios específicos (ej. **UsuarioRepository**, **ConstanciaRepository**) que interactúan con la base de datos . Sin embargo, el diagrama de clases solo muestra las entidades de dominio y no incluye las clases de persistencia o los repositorios que el desarrollador ha proyectado para la arquitectura .

Elementos faltantes

1. **Falta de método de Login: Docente** y **Administrativo** no pueden autenticarse según el diagrama de clases(**iniciarSesion()),** invalidando el flujo principal de acceso.
2. **Entidad "Administrador" huérfana:** En los componentes no hay un controlador que ejecute las acciones de la clase **AdministradorDDA** .

3. **Método de Registro:** Falta el método **RegistrarUsuario()** para ejecutar el **RF-01 (Registro de perfil)** en la capa de clases .
4. **Trazabilidad de Folios:** El componente **ConstanciaRepository** sugiere el manejo de folios y fechas, lo cual coincide con los atributos de la clase **Constancia**, pero no hay un método en **Inscripcion** que dispare la creación automática tras validar el 100% de horas, solo se menciona en el flujo pero no en la firma de métodos.
5. **Falta de Repositorios en Clases:** El diagrama de componentes proyecta el uso de **Repositories**, pero estas clases no fueron modeladas en el diagrama de clases, creando una brecha entre la estructura de datos y la arquitectura física.

Observaciones técnicas claras

- **Nombres Inconsistentes:** Se recomienda estandarizar el nombre del actor/clase. En casos de uso se denomina "Administrador" y en clases "AdministradorDDA" . Aunque el concepto es el mismo, en diseño arquitectónico debe haber una correspondencia exacta.
- **Separación de Responsabilidades:** Se observa una correcta segregación en el diagrama de componentes (UI → Controller → Service → Repository) , lo cual cumple con los lineamientos de no mezclar lógica de negocio en controladores o repositorios.
- **Riesgo de Acoplamiento:** El servicio de inscripciones depende directamente del servicio de cursos para validar el cupo. Esto es correcto, pero se debe asegurar que se implemente mediante interfaces para evitar el acoplamiento excesivo mencionado en los riesgos técnicos.